

Extracting Random Bits From Noise In Temperature Sensors

Paul J. Han

The University of Texas at Austin
{pjh2235}@eid.utexas.edu

Abstract

In embedded systems and modern computing in general, generating random numbers is a key issue for security. While there are pseudorandom number generators, these can result in repeated numbers and with the same seed, are deterministic. As a result, for some security tasks, it is not suitable as they can be repeated and exploited. On the other hand, truly random number generators are derived from physical phenomena as a source of entropy. However, these hardware based RNG rely upon external hardware to measure from sources of entropy, adding to the cost of manufacturing. This issue becomes more prevalent in embedded systems which are meant to be very low cost and functional. Using a common microcontroller board such as the EX-TM4C123GXL and a 100k ohm thermistor, this project explores whether temperature sensors can be used as a source of true randomness and if so, the bandwidth of bits generated. The results suggest that while a temperature sensor under ambient conditions can be utilized to generate random bitstreams, a malicious attacker could introduce a constant heat source, resulting in deterministic bitstreams. This would make generating bitstreams by the attacker exceedingly simple, compromising the numbers generated for cryptographic purposes.

1 Introduction

There are two types of random number generators. Pseudo random number generators generate seemingly random numbers, but are actually deterministic as they can only approximate randomness. True random number generators generate genuinely random numbers, usually by relying on measurement of physical phenomena such as Johnson-Nyquist noise from heat [1] [2]. Truly random number generators are required for applications such as strong encryption keys in secure systems [3].

For robust SoCs meant for more intensive tasks such as modern CPUs, there is hardware built into the silicon that measures physical phenomena and exposes these random bits to the programmer through the ISA. For example, in x86-64, Intel and AMD expose random numbers to the programmer using the RDSEED instruction in formats of 16/32/64 bits [4]. Other ISAs also deliver these random numbers to the programmer. Other examples of random number generators include Cloudflare's lava lamp infrastructure where a wall of 100 lava lamps are photographed periodically. As the

bubbles in lava lamps do not take the same structure deterministically and are truly random, the photograph's pixels being assigned values represent these truly random numbers. These values are then used to generate strong cryptographic keys for OpenSSL and other important encryption purposes [5].

However, these examples all require significant infrastructure and can be costly. Whether the cost comes from the space and lava lamps or the cost of the added footprints to modern silicon, there is a significant cost associated with the generation of truly random numbers. On the other hand, embedded systems are typically associated with cost effectiveness while still requiring the generation of truly random numbers for security purposes.

In an effort to address the constraint of truly random number generation while maintaining the cost effectiveness required in embedded systems, this project proposes the generation of random bits by measuring thermal noise in an extremely cheap thermistor connected to a microcontroller via sampling from an analog-to-digital converter (ADC). This thermistor can be obtained cheaply on modern retailers for approximately \$2.5 per thermistor while micro-controllers such as the TM4C1236HGP, its corresponding breakout board, and similar micro-controllers can be obtained for \$13 on most retailers. The total cost of this system comes out to \$15.5, being significantly more cost effective than the added footprint cost of silicon or a wall of lava lamps. By obtaining the least significant bit of the sampled ADC value which is the most subject to noise as the randomly generated bit, we can generate random bits at a bandwidth limited only by the sampling rate of the ADC.

The results of this project confirm that these cheap components can be used as a source of truly random bits and to confirm that the bandwidth is limited only by how often the ADC value can be sampled from the thermistor. However, this is not without drawbacks as the presence of a high temperature heat source in close proximity to the thermistor results in repeated bitstreams, making the generated numbers deterministic. This reduces the effectiveness of noise in temperature sensors (Johnson-Nyquist noise) as sources of entropy as it is possible that malicious entities with physical access to the sensor could apply a high temperature heat source and generate repeated bits.

To summarize,

- Truly random number generators in embedded systems are limited by their cost, either by added hardware in the silicon or by the required infrastructure. However, truly random number generators are still required in embedded systems.
- To address this issue, this project proposes connecting thermistors to a microcontroller to investigate the measurement of thermal noise as a source of entropy and thus, random bits.
- The results confirm the use of thermal noise as a source of random bits, but also indicate that the presence of a high temperature heat source in close proximity to the sensor can generate a repeated bitstream, compromising the use of temperature measurements as a source of entropy.

2 Background

There has been significant work in developing true random number generators for computing. A famous example of true random number generation is the wall of lava lamps used by Cloudflare. A camera takes periodic pictures of the wall of lava lamps. As a lava lamp never takes the exact same shape, a random number is extracted from the periodic images as each pixel in the image is in a sequence of numbers. These true random numbers are used for applications such as generating keys in OpenSSL and other cryptography related to security [5]. Other examples include hardware included on x86-64 processors by AMD and Intel that extract entropy and expose an instruction called RDSEED which generates a 16/32/64 bit random integer [4]. However, these applications all require extensive hardware either built into the SoC or the physical infrastructure of lava lamps to measure from high entropy sources. In embedded systems, however, sensors are used to measure from truly random sources of entropy such as temperature sensors, system clocks, timing of user inputs on a button, and clock jitter [1]. This project aims to investigate if a sensor such as a thermistor which can be obtained easily at the EER check out desk can also be used as a reliable measure of entropy and its bandwidth.

3 Threat Model

As part of our threat model to the performance of these sensors, we assume that an attacker has physical access to the sensor and can experiment with the sensor as desired. This includes being able to apply a constant heat source directly to the thermistor and influence the generation of bits. The attacker also has knowledge that the thermistor is the source of randomly generated bits for cryptographic purposes such as the generation of OpenSSL keys in this system and desires to influence the generation of these bits by manipulating the sensor.

4 Proposed System Design

The system design for this experiment is simple. As can be seen in Figure 1, there is a 100k ohm thermistor connected to the ADC input pin for a microcontroller breakout board. There is also a resistor in series with the same resistance as the thermistor, creating a voltage divider circuit. As the voltage source and resistance is fixed, the changes in resistance from the thermistor from temperature changes result in voltage fluctuations which can be measured by the microcontroller. The ADC input pin samples at its maximum bandwidth and measures the least significant bit of the ADC value as it theoretically is altered the most often, giving the temperature measurement the most entropy. The bits measured are sent over UART to a host computer so that the bits generated can be inspected for further analysis. With this design, the theoretical bandwidth of entropy is the maximum sampling frequency of the ADC.

5 Experimental Setup

```

1 void Timer3A_Handler(void) {
2     TIMER3_ICR_R = TIMER_ICR_TATOCINT; //
3     acknowledge TIMER3A timeout
4     if(Flag != 0){
5         return;
6     }
7     Data = ADC0_InSeq3();
8     Flag = 1;
9 }

```

Listing 1. Timer-based interrupt for ADC sampling

```

1 void itoa(uint32_t n, char * buffer){
2     uint32_t num = n;
3     int count = 0;
4     while(num != 0){
5         count++;
6         num /= 10;
7     }
8     num = n;
9     buffer[count] = '\0';
10    count--;
11
12    for(int i = count; i >= 0; i--){
13        uint8_t digit = num % 10;
14        buffer[i] = (char)(digit + 0x30);
15        num /= 10;
16    }
17 }

```

Listing 2. Integer-to-ASCII helper function

The experiment is set up with a TM4C123GH6PM microprocessor embedded into a breakout board, specifically the EX-TM4C123GXL as can be seen in Figure 2. The ADC receives input from the voltage divider circuit consisting of a thermistor and a resistor of the same resistance, 100k ohms. Utilizing the PE4 ADC pin and timer based interrupts in software, samples from the ADC are taken at a frequency of 100 kHz. As the TM4C123GH6PM chip is capable of sampling

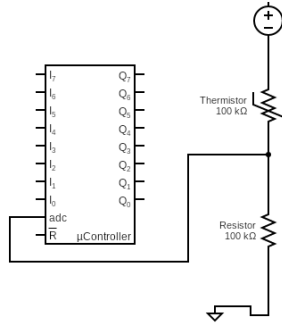


Figure 1. System design for a generic microcontroller and voltage divider circuit

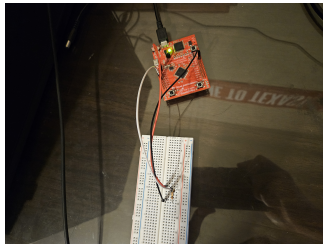


Figure 2. Experimental setup with EX-TM4C123GXL break-out board and voltage divider circuit with thermistor

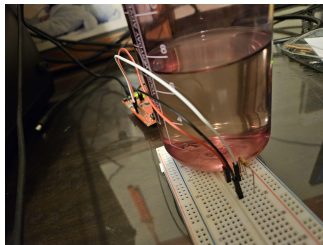


Figure 3. Experimental setup with a heat source

as fast as timer-based interrupts can fire in the system, the sampling frequency was chosen to be 100 kHz. At a bus frequency of 80 MHz, 100 kHz can be achieved with 800 cycles in between interrupts. As there is code executed in the main loop - coordinated using a mailbox - between each interrupt, this gives sufficient cycles to finish before another interrupt begins. The interrupt handler can be seen in Listing 1. This code includes helper functions to send temperature data to another PC via UART, bit extraction, and converting an integer to an ASCII array (itoa) which can take many cycles. An example of a helper function (itoa) can be seen in Listing 2. The bit generated will be the least significant bit of this sampled ADC value.

To follow the threat model, the experiment will occur in two steps. One step will measure the generation of bits by measuring ambient temperature through the thermistor. The

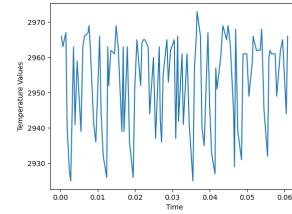


Figure 4. Temperature data at ambient temperature

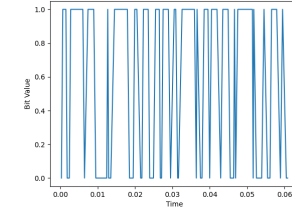


Figure 5. Bits generated at ambient temperature

next step will measure the generation of bits when the thermistor temperature is being manipulated using a **constant heat source**. This constant heat source is the presence of a plastic water bottle with boiling hot water inside of it. As water retains heat quite steadily and this heat diffuses through plastic, the placement of this water bottle right next to the thermistor models an attacker placing a constant heat source on the thermistor to manipulate its values. An example of this system can be seen in Figure 3

As these bits are generated, they are sent over UART to a host computer for further analysis along with the temperature value. This analysis ensures that the bits generated are actually random instead of being incorrect. For example, an obviously incorrect bit generator could return all 0s or 1s. Another example of incorrect behavior could be that the ADC value never fluctuates, demonstrating that these thermistors cannot be used to generate random bits. This overall system is placed in a room temperature environment with no fans or obvious fluctuating temperature source beyond a typical A/C system in an indoor environment.

6 Results

After setting up the circuit as seen in 2, the temperature data and bits generated were sent to a host computer for further analysis, generating Figures 4 and 5. As can be seen in the highly fluctuating values in Figure 4, the temperature data has significant noise in it, confirming the presence of Johnson-Nyquist noise in the ambient temperature measured from a thermistor. With this result, I can confirm that temperature can be a source of entropy for the generation of random bits. Moreover, temperature data notwithstanding, the results confirm that the bits generated from ambient

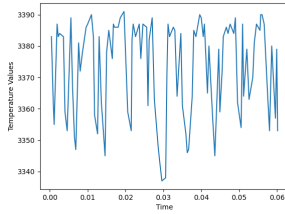


Figure 6. Heat data with high temperature source applied

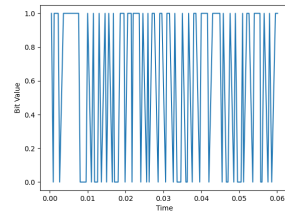


Figure 7. Bits generated with high temperature source applied

temperature are random as can be seen in Figure 5. The bandwidth of bits generated can be calculated as the frequency of the interrupts sampling the ADC. As the interrupt triggers at 100 kHz frequency, the ADC is sampled at 100 kHz as well, resulting in a bandwidth of 100k bits per second. As many sources of entropy can have significantly reduced bandwidth, this is significantly improved bandwidth.

However, following our threat model, I placed an extremely hot water bottle with boiling water and applied it directly to the thermistor to model an attacker applying a high temperature heat source next to the thermistor. As can be seen in Figure 6, this results in more consistent measurements with the temperature measurements remaining at its peak, rather fluctuating within a narrow range as it would with the presence of noise. This subsequently results in significant amounts of bits repeating when bits are being generated as can be seen in Figure 7. As the heat source is applied, 1s bits are generated at a significantly higher rate than 0s. With the application of a sufficiently high temperature heat source, *only* 1s could be generated or enough 1s that 0s could be determined, resulting in a bit stream that is deterministic and manipulated by the attacker.

The results indicate that a temperature sensor as cost-effective as a thermistor can measure Johnson-Nyquist noise that is proven random, providing an entropy source that is capable of generating truly random bits. Given a given microcontroller sampling the ADC and therefore, temperature at a frequency of 100 kHz, this system can achieve a bandwidth of random bits generated of 100k bits per second. However, this is not without drawbacks as an attacker with physical

access to the thermistor can manipulate its results to generate a high number of 1s by applying a sufficiently high temperature heat source to the thermistor, rendering the generation of random bits deterministic. While the cost-effective, simple nature of these components is promising, there may be further sources of entropy that meet the design requirements of cost-effectiveness and true randomness without being vulnerable to physical manipulation by attackers.

7 Conclusion

In this project, to mitigate the burden of cost in generating truly random bits for embedded systems, I explored the use of a thermistor in a voltage divider circuit attached to a microcontroller. As embedded systems have to be cost-effective compared to other infrastructure such as Cloudflare's lava lamp and still generate random bits for tasks such as key generation, the simplicity of this design was crucial. These components were cost-effective, retailing at \$15 for the entire system and measured Johnson-Nyquist noise from a thermistor which is proven to be random and therefore a sufficient source of entropy for generating random bits. In addition to their randomness, this design had to generate bits at a bandwidth sufficient to meet the needs of key generation which can require hundreds of bits. Within these constraints, I discovered that measuring Johnson-Nyquist noise can be a sufficient source of entropy and at a high bandwidth within a EX-TM4C123GXL microcontroller connected to a thermistor by ADC sampling. However, modeling an attacker with physical access to this device, the application of a high temperature heat source to the thermistor caused the bits generated to repeatedly be 1s. This compromised its ability to generate random bits and could be a cause of concern if this design were to be deployed. Further exploration of different entropy sources which cannot be physically manipulated by an attacker is needed.

References

- [1] Ben Lampert, Riad S. Wahby, Shane Leonard, and Philip Levis. Robust, low-cost, auditable random number generation for embedded system security. *Proceedings of the 14th ACM Conference on Embedded Network Sensor Systems CD-ROM*, 11 2016.
- [2] J F Qu, S P Benz, H Rogalla, W L Tew, D R White, and K L Zhou. Johnson noise thermometry. *Measurement Science and Technology*, 30:112001, 09 2019.
- [3] Parul Sharma, Syed Farah Naz, and Ambika Prasad Shah. True random number generators in modern cryptography: A review. *Memories - Materials, Devices, Circuits and Systems*, 13:100138, 06 2026.
- [4] John P. Mechalias. Intel® digital random number generator (drng) software implementation..., 10 2018.
- [5] Cloudflare. How do lava lamps help with internet encryption?, 2024.